

PhysAgent: A Multi-Agent Approach to Automated Discovery of Physical Laws

Xiao-Qi Han^{1,2}, Ze-Feng Gao^{1,2,*}, Peng-Jie Guo^{1,2}, Zhong-Yi Lu^{1,2,3,*}

¹School of Physics and Beijing Key Laboratory of Opto-electronic Functional Materials & Micro-nano Devices, Renmin University of China, Beijing 100872, China

²Key Laboratory of Quantum State Construction and Manipulation (Ministry of Education), Renmin University of China, Beijing, China

³Hefei National Laboratory, Hefei 230088, China

*Corresponding authors

Abstract

The discovery of physical laws has traditionally relied on human intuition, analytical reasoning, and experimental observation. However, modern physics research is increasingly constrained by challenges such as high specialization, fragmented workflows, and limited computational resources, which impede scientific progress. To address these issues, we introduce **PhysAgent**, a novel multi-agent system powered by large language models (LLMs), designed to autonomously execute end-to-end scientific workflows—from hypothesis generation and computational modeling to data analysis and discovery. PhysAgent’s innovation lies in its specialized agent collaboration: a Mentor Agent guides scientific reasoning through Socratic questioning, a Student Agent handles technical execution (e.g., code implementation, DFT calculations), and a Leader Agent dynamically optimizes task scheduling and resource allocation. Integrating domain-specific tools like first-principles simulations (e.g., Quantum ESPRESSO, VASP), PhysAgent ensures reproducibility while maintaining human-in-the-loop refinement. We demonstrate its capability to autonomously derive physical laws—such as Kepler’s laws from orbital data and Newton’s second law from force-motion experiments—without prior knowledge. Furthermore, it extends to ab initio materials modeling, automating electronic structure calculations (e.g., GaAs band gaps). In addition, PhysAgent simulates complex real-world phenomena, such as raindrop flow on train windows, highlighting its adaptability beyond traditional physics problems. By harmonizing LLM-driven planning with domain-specific tools, PhysAgent establishes a trustworthy, scalable paradigm for AI-driven physics research, highlighting the transformative potential of multi-agent intelligence in accelerating discovery across classical and quantum systems.

Keywords: multi-agent system, large language models, first-principles simulations, automated scientific discovery

Introduction

Physics plays a central role across diverse domains, including energy [1], materials [2–5], information [6], and life sciences [7]. In recent years, advances in experimental techniques and computational capabilities have led to an unprecedented surge in data volume [8–11], model complexity [12–14], and interdisciplinary challenges [15]. However, the progress of physics research is increasingly constrained by high specialization, fragmented workflows, and limitations in computational resources. From theoretical model development and numerical simulation [16–18] to experimental

validation, modern physics research demands the integration of multidisciplinary knowledge [19–21], the orchestration of intricate workflows [22–24], and exhaustive exploration of vast parameter spaces. The growing disparity between the complexity of physical systems [25] and the limited cognitive and computational bandwidth available to researchers highlights the urgent need for a new research paradigm—an intelligent system capable of integrating knowledge, autonomously exploring models, and accelerating scientific discovery.

In recent years, the rapid advancement of large language models (LLMs), such as GPT-4 [6] and DeepSeek [26], has led to the emergence of AI agents [27–30]—autonomous or semi-autonomous systems powered by LLMs as their central reasoning engines. Representative frameworks include MetaGPT [28], which model software engineering processes through role-based agent collaboration; AutoGen [29], which enables customizable multi-agent conversations with enhanced reasoning capabilities; and LangChain [30], a widely adopted toolkit for building LLM-based agents that can interact with tools, APIs, and memory. These AI agents are profoundly transforming multiple scientific and engineering domains. In particular, fields such as law [31], biomedicine [32], software engineering [28, 33], and materials science [34] have witnessed significant progress, as these agents automate repetitive tasks, greatly enhance research efficiency, and enable breakthroughs that were previously considered unattainable. As a result, AI agents are becoming indispensable tools for accelerating scientific discovery and engineering innovation.

However, existing agent frameworks (such as MetaGPT [28] and AutoGen [29]) face significant limitations in the context of physics research. They lack deep causal reasoning capabilities grounded in physical theory [35, 36], are unable to natively integrate specialized tools like first-principles simulations, and struggle with dynamic task scheduling across multiple scales [37]. Specifically, these systems often rely on predefined operation sequences (e.g., parameter sweeps [38, 39]), making it difficult to construct goal-oriented problem-solving workflows dynamically. Moreover, their limited integration with the physics computing ecosystem (e.g., Linux/Python/MPI) hinders seamless execution from electronic structure [22] calculations to molecular dynamics [37, 40, 41]. Critically, most frameworks lack built-in mechanisms for enforcing physical consistency, such as energy conservation checks, which may lead to unphysical results. Overcoming these challenges requires the integration of three key capabilities: scientific agents with causal reasoning, execution frameworks natively compatible with domain-specific computing platforms, and safety mechanisms that enforce physical consistency.

In this work, we introduce **PhysAgent**, a multi-agent system powered by LLMs [6, 26, 42], designed to autonomously execute end-to-end scientific workflows—from hypothesis generation and computational modeling to data analysis and discovery. PhysAgent’s core innovation lies in its specialized agent collaboration: a Mentor Agent guides scientific reasoning via Socratic questioning, a Student Agent handles technical execution (e.g., code implementation, density functional theory (DFT) calculations), and a Leader Agent dynamically optimizes task scheduling and resource allocation. This triad operates within a physics-grounded paradigm, integrating first-principles simulations (e.g., Quantum ESPRESSO ¹, VASP ²) and rigorous data-driven validation to ensure reproducibility. Crucially, PhysAgent achieves autonomous scientific discovery without prior knowledge of physical laws—demonstrated by its derivation of Kepler’s laws purely from orbital data (Figure 2) and Newton’s second law from force-motion experiments (Figure 3). It further extends to ab initio materials modeling, automating electronic structure calculations (e.g., GaAs band gaps, Figure 4) while maintaining human-in-the-loop refinement. In addition, PhysAgent can simulate complex real-world phenomena, such as raindrop flow on train windows, demonstrating

¹<https://www.quantum-espresso.org/>

²<https://www.vasp.at/>

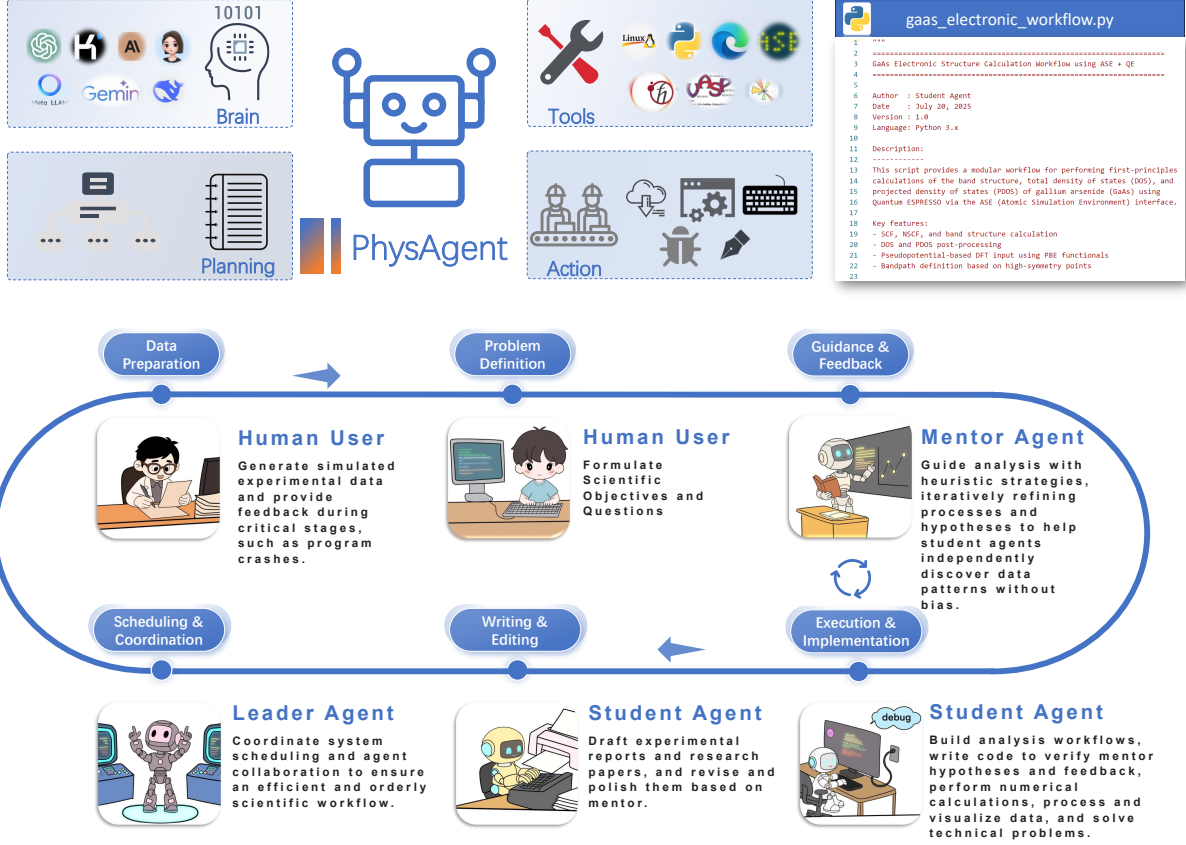


Figure 1: Architecture and Workflow of the PhysAgent System. Schematic of the PhysAgent system, powered by an LLM acting as the agent’s brain, enabling experimental planning, tool usage, and action execution via API calls to models like GPT-4 [6], KIMI [42], Doubao, Llama [43], and DeepSeek [26]. The multi-agent system integrates human users, who provide data and define problems, with specialized agents: the Mentor Agent guides hypothesis formation, the Student Agent handles computation and coding, and the Leader Agent optimizes task scheduling. The figure illustrates a material’s electronic structure calculation script completed through multi-round agent collaboration.

its versatility beyond traditional physics problems (Figure 5). By harmonizing LLM-driven planning with domain-specific tools, PhysAgent establishes a new paradigm for trustworthy, scalable AI-driven scientific exploration.

Results

Architecture and Workflow of the PhysAgent System

As depicted in Figure 1, the PhysAgent system is powered by an LLM that serves as the agent’s brain, enabling capabilities such as experimental planning, tool usage, and action execution. This brain is implemented through direct API calls, supporting a range of LLMs, including GPT-4 ³,

³<https://openai.com/chatgpt/overview/>

Kimi ⁴, Doubao ⁵, Llama ⁶, and DeepSeek ⁷. Similar to existing agents [27], PhysAgent is also equipped with capabilities such as learning [44, 45], reasoning [46, 47], memory [48, 49], reward [50], perception [51], and action [52].

At the initiation of a task, PhysAgent autonomously manages project planning, tool installation, and action execution. These actions encompass operating Linux systems, writing and executing Python scripts, performing web searches and downloads, debugging code, conducting data analysis, invoking software packages such as Quantum ESPRESSO or VASP for electronic structure calculations of materials, and drafting experimental reports and academic papers. The PhysAgent multi-agent system integrates human users and specialized agents, each tasked with distinct responsibilities within the scientific workflow, the detail of these prompt can be see in Supplementary Materials (SM).

- **Human User:** Provides experimental data and defines scientific problems
- **Mentor Agent:** Guides hypothesis formation using Socratic questioning and bias mitigation strategies
- **Student Agent:** Performs numerical computation, code implementation, and technical problem-solving
- **Leader Agent:** Optimizes task scheduling via dynamic resource allocation

In the scientific workflow of PhysAgent, collaboration among multiple agents enables task completion. The Human user defines the scientific objective and research question, generates simulated experimental data, and provides feedback at key stages. The Mentor Agent guides the analytical process using heuristic strategies, formulates hypotheses, and offers feedback to the Student Agent to facilitate autonomous pattern discovery while avoiding prior biases. The Student Agent is responsible for building analysis pipelines, implementing and verifying code, performing numerical simulations, processing and visualizing data, and drafting reports and research papers. As illustrated in Figure 1, the Mentor Agent and Student Agent engage in multiple rounds of feedback and iterative refinement throughout the analytical process, ensuring continual hypothesis improvement and enhanced data interpretation. The Leader Agent manages system resources and orchestrates agent collaboration to ensure efficiency and order. With a closed-loop design encompassing problem formulation, data generation, analysis execution, manuscript writing, and workflow coordination, PhysAgent provides an end-to-end intelligent framework for scientific discovery, offering a new paradigm for solving complex scientific problems.

Here, Figure 1 illustrates an example of materials electronic structure calculations (GaAs) collaboratively completed by PhysAgent. Compared to traditional single-agent models and general multi-agent systems, PhysAgent demonstrates significant advantages in handling complex, multi-step, cross-scale physics research tasks through clear role division, effective knowledge sharing, and efficient strategy coordination. Experimental results show that the system can dynamically construct complete problem-solving workflows—from structure modeling to final property analysis—seamlessly integrate specialized toolchains such as ASE and DFT software, and produce key scientific outputs that meet research standards, including structural models, convergence curves, charge density distributions, and band structure/density of states plots. This capability effectively

⁴<https://www.kimi.com/>

⁵<https://www.doubao.com/chat/>

⁶<https://www.llama.com/>

⁷<https://chat.deepseek.com/>

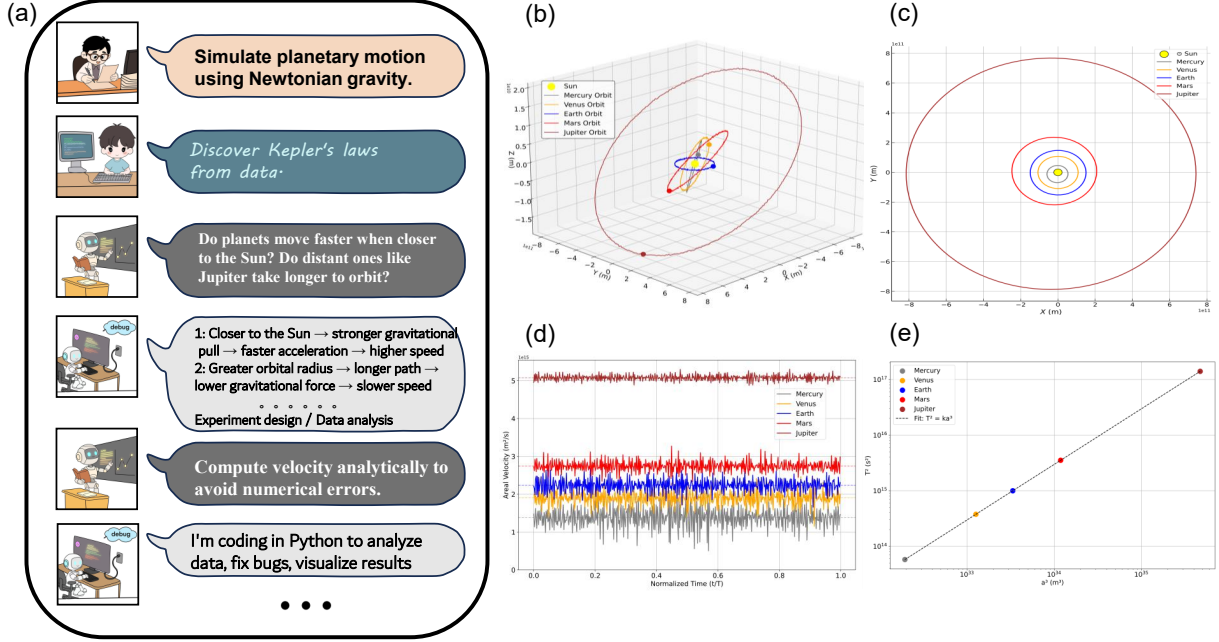


Figure 2: PhysAgent discovers Kepler’s laws of planetary motion. (a) Collaborative workflow of PhysAgent discovering Kepler’s planetary motion laws: Human-generated time-series three-dimensional trajectory data for five solar system planets (Mercury, Venus, Earth, Mars, and Jupiter) were analyzed through mentor-student agent collaboration in the PhysAgent system. The mentor agent provided heuristic questions to guide the student agent in exploring orbit shapes, velocity variations, and period-radius relationships, while the student agent implemented code and resolved technical challenges, producing publication-quality visualizations. (b) Planetary Motion Trajectories (Human User): Time-series 3D trajectories of five planets in the solar system—Mercury, Venus, Earth, Mars, and Jupiter—generated by human users using numerical methods. (c) Orbit Shape Visualization (Student Agent): Projections of the orbits of Mercury, Venus, Earth, Mars, and Jupiter in the XY plane, distinguished by colors (gray, orange, blue, red, brown). The orbits are elliptical with the Sun at one focus, confirming Kepler’s First Law. (d) Areal Velocity Analysis (Collaborative Result): Areal velocity variations over time for each planet, using the same color scheme. The approximately constant areal velocity indicates that the line connecting a planet to the Sun sweeps equal areas in equal times, consistent with Kepler’s Second Law. (e) Period-Radius Relationship (Collaborative Result): Relationship between the square of the orbital period (T^2) and the cube of the semi-major axis (a^3), with planets distinguished by the same color scheme. A linear fit demonstrates $T^2 \propto a^3$, validating Kepler’s Third Law.

overcomes the limitations of conventional frameworks in causal reasoning and integration of specialized tools, substantially enhancing adaptability, robustness, and scalability in complex scientific computations.

PhysAgent discovers Kepler’s laws.

To demonstrate the capability of PhysAgent in autonomously discovering scientific laws, we selected a more challenging task—Kepler’s laws of planetary motion. Similar to previous settings, PhysAgent is required to infer the underlying principles of planetary motion solely from simulated experimental data, without any prior knowledge of Kepler’s laws. As shown in Figure 2(a), the task is initiated by a human user who numerically simulates the time-series three-dimensional trajectories of five planets in the solar system (Mercury, Venus, Earth, Mars, and Jupiter) under the gravitational influence of the Sun. The user then poses a scientific objective: Is it possible to discover Kepler’s three laws of planetary motion from these data?

In the PhysAgent multi-agent system, multiple agents collaboratively derived Kepler’s laws of

planetary motion. As shown in Figure 2 (a), the mentor agent employed heuristic questioning to guide the student agent in exploring planetary motion from various perspectives. For instance, in analyzing orbit shapes, the mentor agent asked, “What is the shape of planetary orbits around the Sun? Circular or elliptical?” In examining velocity variations, it inquired, “Is the orbital speed of a planet constant? Is it faster when closer to the Sun or when farther away?” In deriving the relationship between period and distance, it posed, “Do planets farther from the Sun, such as Jupiter, require more time to complete an orbit?” These questions stimulated critical thinking in the student agent. Based on prior knowledge, the student agent formulated hypotheses: stronger gravitational forces near the Sun lead to greater acceleration and higher planetary velocities; weaker forces farther from the Sun result in slower velocities; and planets with larger orbital radii, such as Jupiter, have longer orbital paths and thus extended periods. Using Python tools, the student agent designed experiments to analyze time and position data (`time`, `x`, `y`, `z`) from the dataset, computing orbital shapes, velocity variations, and orbital periods to test these hypotheses. During the experiments, the student agent addressed challenges such as data noise and numerical computation errors and utilized visualization tools to generate orbital plots, areal velocity curves, and period-radius relationship graphs.

As shown in Figure 2 (c), the orbits of Mercury, Venus, Earth, Mars, and Jupiter are elliptical, with the Sun at one focus, confirming Kepler’s First Law. As depicted in Figure 2 (d), the areal velocity of each planet is approximately constant, indicating that the line connecting a planet to the Sun sweeps equal areas in equal times, consistent with Kepler’s Second Law. As illustrated in Figure 2 (e), the square of a planet’s orbital period is proportional to the cube of its semi-major axis, validating Kepler’s Third Law. Throughout the collaborative process, the mentor agent engaged in multiple rounds of interaction with the student agent, providing feedback to refine hypotheses (e.g., adjusting from circular to elliptical orbits), optimize analytical methods (e.g., enhancing the precision of areal velocity calculations), and improve visualization designs to clearly highlight scientific patterns. Ultimately, the PhysAgent system, relying solely on observational data, successfully derived Kepler’s three laws, fully replicating the scientific discovery process from observation to hypothesis, experimental validation, and conclusion, thereby demonstrating the efficiency and exploratory capability of multi-agent collaboration.

PhysAgent discovers Newton’s Second Law.

To further evaluate the capability of the multi-agent system PhysAgent in autonomously discovering fundamental physical laws, we designed a classical mechanics task that challenges the system to derive the quantitative relationship between force and motion without any prior knowledge of Newton’s second law. This task not only assesses PhysAgent’s data analysis capacity but also emphasizes a discovery process that begins entirely from first principles. As shown in Figure 3 (a), the task begins with a human user who generates simulated position-time data of a cart moving on an inclined plane, followed by posing the question: can one discover how force influences motion from these data? Subsequently, multiple agents within the PhysAgent system collaboratively engaged in the task. The mentor agent proposed scientific strategies and guided the student agent in uncovering patterns from the data, emphasizing the avoidance of any prior knowledge of physical laws to ensure objectivity in the derivation. Next, the student agent constructed the analysis workflow from scratch, including designing the data collection scheme, formulating reasonable hypotheses, processing the experimental data, and conducting visualization and curve fitting using Python and standard scientific libraries such as *NumPy*, *Pandas*, and *Matplotlib*. The human user provided iterative feedback at key stages, particularly with respect to visualization standards, graphical presentation, and scientific writing norms. Throughout this collaborative process, the mentor and

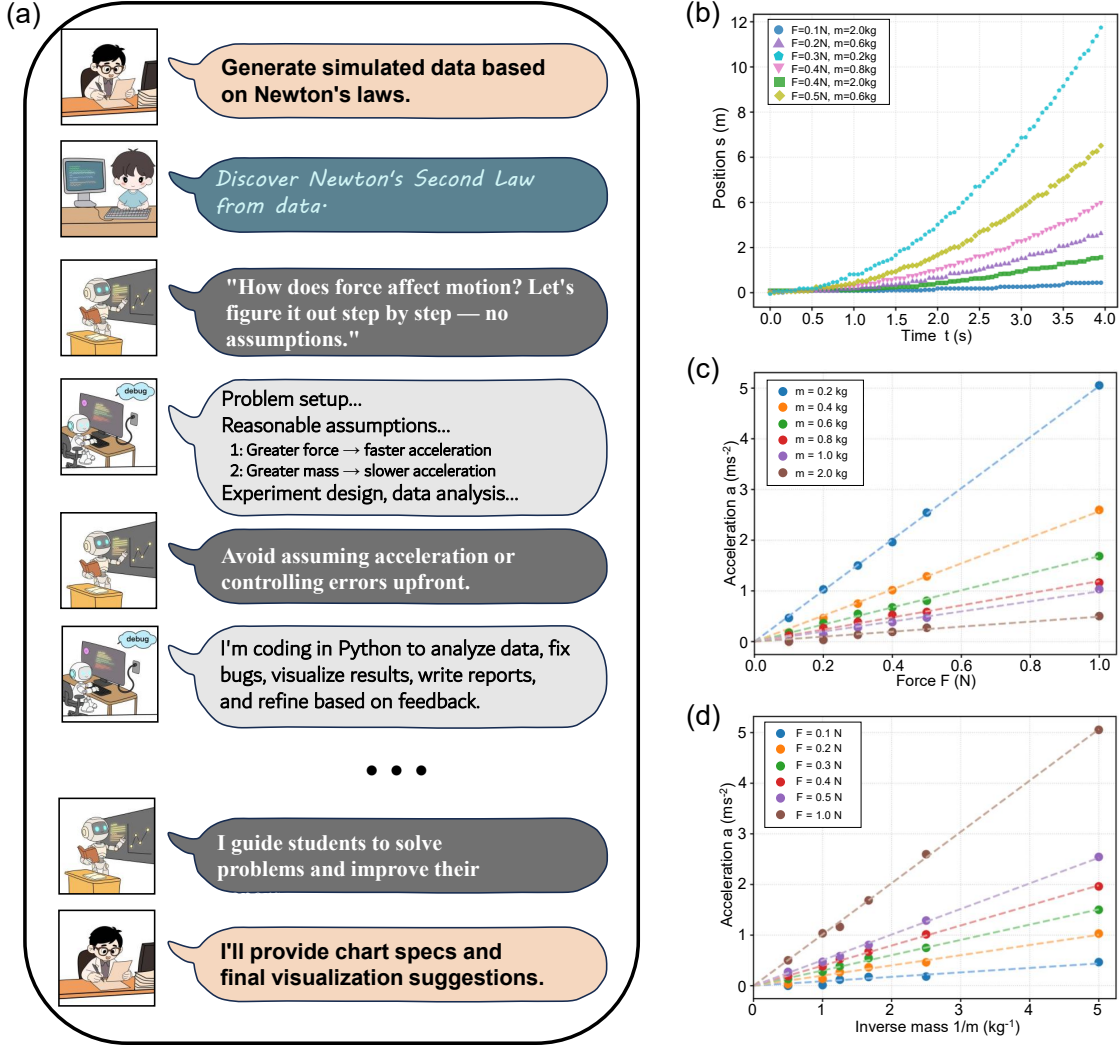


Figure 3: PhysAgent discovers Newton’s Second Law. (a) Collaborative workflow of PhysAgent discovering Newton’s second law: Human-generated position-time data of a cart on an inclined plane were analyzed through mentor-student agent collaboration. The mentor provided analysis strategies while the student implemented code and resolved technical challenges. Final visualizations were refined for publication standards. (b) Position-time visualization (Student agent): Position s versus time t under varying force-mass conditions. Distinct markers/colors represent different (F, m) pairs. Parabolic trajectories with added measurement noise confirm uniformly accelerated motion. (c) Acceleration-force relationship (Collaborative result): Linear a - F dependence derived through iterative optimization. Color-coded masses demonstrate proportionality $a \propto F$ at constant m . (d) Acceleration-inverse mass relationship (Collaborative result): Linear a - $1/m$ relationship established via agent collaboration. Color-coded forces verify proportionality $a \propto 1/m$ at constant F .

student agents engaged in multiple rounds of interaction, continuously refining their hypotheses and improving the analysis workflow. Ultimately, the agents successfully derived Newton’s second law directly from the simulated data, completing the discovery process entirely from observational evidence.

Figure 3 (b) illustrates the trajectories of a cart under six different combinations of applied force F and mass m : $(F = 0.1\text{ N}, m = 2.0\text{ kg})$, $(F = 0.2\text{ N}, m = 0.6\text{ kg})$, $(F = 0.3\text{ N}, m = 0.2\text{ kg})$, $(F = 0.4\text{ N}, m = 0.8\text{ kg})$, $(F = 0.4\text{ N}, m = 2.0\text{ kg})$, and $(F = 0.5\text{ N}, m = 0.6\text{ kg})$. The cart’s position s exhibits an approximately parabolic dependence on time t , indicating uniformly

accelerated motion. To better mimic experimental conditions, small amounts of noise were added to the data. This visualization provides the foundation for the subsequent derivation of physical quantities such as acceleration.

Under the guidance of the mentor agent, the student agent computed the acceleration a from the position-time data via numerical differentiation. As shown in Figure 3 (c), the acceleration of the cart under varying forces is plotted for fixed mass values. Each color represents a different mass ($m = 0.2, 0.4, 0.6, 0.8, 1.0$, and 2.0 kg). Within each group, a clear linear relationship between acceleration and force is observed, confirming that $a \propto F$ when mass is held constant. Furthermore, Figure 3 (d) presents the variation of acceleration under fixed external force conditions, plotted against the inverse of mass $1/m$. Different colors correspond to different force values ($F = 0.1, 0.2, 0.3, 0.4, 0.5$, and 1.0 N). The results exhibit a strong linear dependence of a on $1/m$, supporting the hypothesis that $a \propto 1/m$. By integrating these two key relationships, PhysAgent successfully derived the quantitative law:

$$a \propto \frac{F}{m} \Rightarrow F = ma$$

without any prior knowledge of physics, relying solely on experimental data and multi-agent collaboration. This marks a significant milestone where PhysAgent autonomously derives the mathematical form of Newton’s Second Law without relying on any prior assumption of the law itself. From problem formulation, data acquisition, visualization, hypothesis construction, and data analysis to law discovery, the entire process closely mirrors the logical path of human scientific inquiry. This demonstrates PhysAgent’s capability for interpretable and verifiable discovery in addressing fundamental problems in science.

PhysAgent performs quantum calculations.

In addition to the automated derivation of classical mechanics laws, PhysAgent also enables fully automated quantum mechanical calculations for materials. To demonstrate its capability, we consider the case of gallium arsenide (GaAs), a direct band gap semiconductor with a zinc blende structure. As shown in Figure 4(a), the task begins with a human user uploading the CIF file of GaAs and requesting an analysis of its electronic structure. The objective is further clarified as: “perform electronic structure calculations of GaAs based on DFT.”

Once the goal is established, the mentor agent searches for DFT workflows and requires online tools, then forwards this information to the student agent. The student agent then autonomously plans and executes a complete first-principles DFT workflow to study the electronic properties of GaAs. As illustrated in Figure 4(a), the workflow starts with setting up the computational environment. The student agent installs and configures the Atomic Simulation Environment (ASE), Quantum ESPRESSO (QE), Python, and relevant visualization packages. It also sets the paths for pseudopotential files and necessary environment variables. During the structure modeling phase (Figure 4(b)), ASE is used to parse the user-provided CIF file and construct the GaAs crystal structure. In the calculation phase, a self-consistent field (SCF) calculation is performed to obtain the ground-state charge density, with the energy convergence process shown in Figure 4(c). This is followed by band structure calculations along high-symmetry paths in the Brillouin zone (Γ –X–W–K– Γ –L–U–W–L–K), and a non-self-consistent field (NSCF) calculation to generate wavefunctions on a dense k -point mesh for density of states (DOS) analysis. The student agent invokes `dos.x` and `projwfc.x` to compute the total density of states (DOS) and the projected density of states (PDOS), respectively.

In the visualization stage, the student agent uses `pp.x` to convert the SCF-derived charge density into CUBE format. Figure 4(e) shows a logarithmic color-mapped slice in the XY plane,

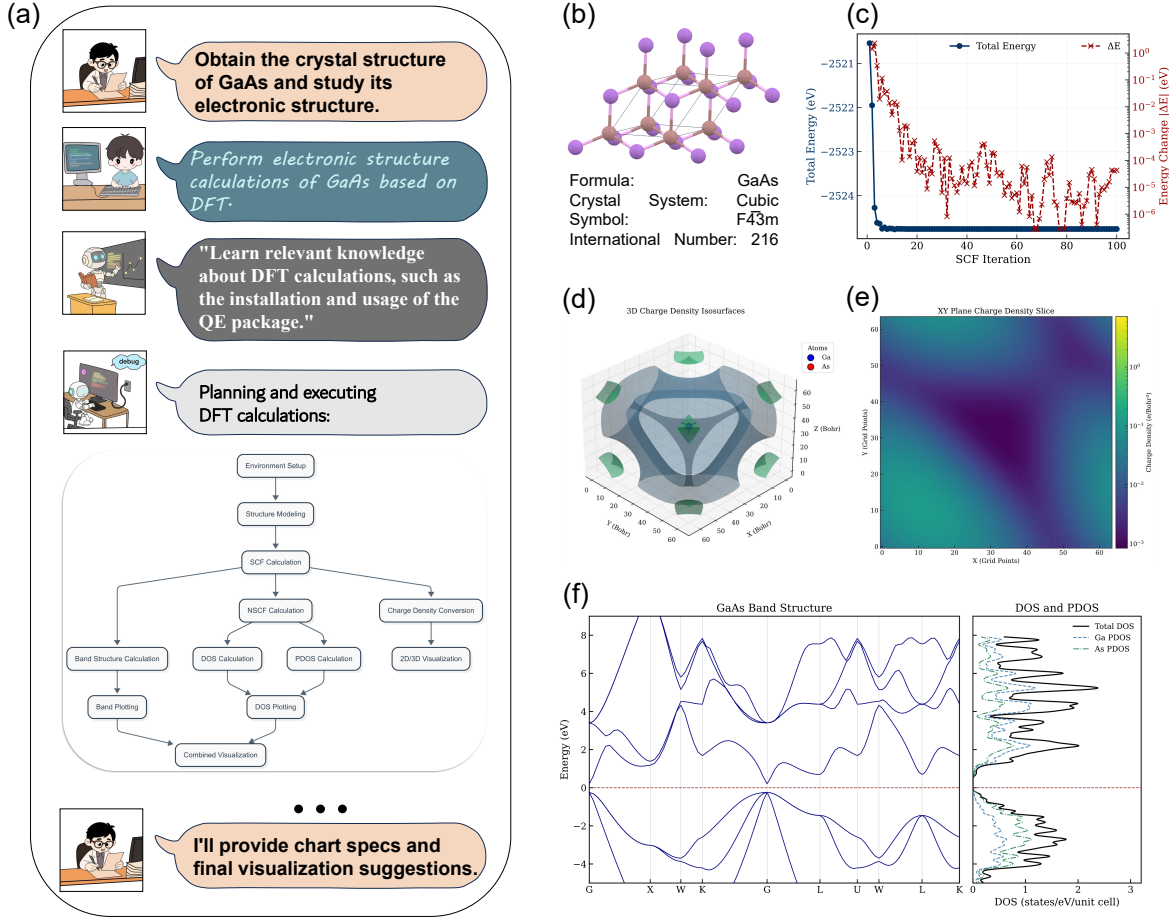


Figure 4: PhysAgent performs quantum calculations. (a) Collaborative workflow of PhysAgent performing DFT electronic structure calculations for GaAs: A human user initiates the task by uploading the CIF structure file of GaAs and requesting a DFT-based electronic structure calculation. The mentor agent retrieves necessary workflows and tools, while the student agent autonomously configures the environment and executes the full DFT workflow using Quantum ESPRESSO and ASE, producing scientifically accurate results. (b) Structure Modeling (Student Agent): The crystal structure of GaAs is parsed from the CIF file and reconstructed using ASE, visualized here as a ball-and-stick model. This step prepares the input for first-principles calculations. (c) SCF Energy Convergence (Collaborative Result): Energy convergence behavior during the self-consistent field (SCF) calculation is plotted, showing systematic reduction of total energy over iterations, confirming convergence to the ground-state charge density. (d) Charge Density Isosurface Visualization (Collaborative Result): 3D isosurfaces of the SCF charge density are rendered, overlaid with atomic positions and lattice vectors. The distribution reflects electronic localization around atomic sites. (e) Logarithmic Charge Density Slice (Collaborative Result): A 2D log-scale slice of the charge density in the XY plane, illustrating fine variations in electron distribution and bonding characteristics. (f) Band Structure and Density of States (Collaborative Result): The left panel shows the band structure of GaAs along high-symmetry paths (Γ -X-W-K- Γ -L-U-W-L-K); the right panel displays total DOS and projected DOS onto Ga and As orbitals. Both panels share a common energy axis for direct comparison, revealing GaAs as a direct band gap semiconductor.

while Figure 4(d) illustrates 3D isosurfaces overlaid with atomic positions and lattice vectors. Furthermore, the agent constructs a combined band structure and DOS plot (Figure 4(f)), where the left panel displays energy dispersion with labeled high-symmetry points and the right panel overlays total and projected DOS for Ga and As atoms. Both panels share a common energy axis for direct comparison. The fully automated pipeline outputs key physical quantities such as

charge density, band eigenvalues, and orbital-resolved DOS, enabling a comprehensive theoretical understanding of the electronic structure of GaAs.

PhysAgent simulates raindrop flow on train windows.

In this section, we apply PhysAgent to simulate and interpret experimental phenomena (Figure 5(a)). The flow of raindrops on a window is a typical thin-film flow problem influenced by gravity, surface tension, viscosity, and external driving forces such as air shear stress and raindrop deposition. When a train is moving at high speed, rainwater on the window often forms obliquely downward flow paths, sometimes with meandering trajectories that branch into tree-like patterns, accompanied by evaporation and dripping (Figure 5(c)-(d)). PhysAgent models this behavior using the thin-film approximation, employing a height evolution equation to describe the spatiotemporal dynamics of the film thickness, and incorporates stochastic raindrop deposition, branching mechanisms, as well as evaporation and dripping effects to reproduce realistic scenarios.

As show Figure 5(b), the model assumes that the film thickness $h(x, y, t)$ is much smaller than the flow length scale (thin-film approximation), that the flow is laminar, and that inertial effects are negligible. The coordinate system is defined with the x -axis along the horizontal length of the window and the y -axis along its vertical height. The flow is primarily driven by gravity and air shear stress, moving obliquely downward. The thin-film flow can be derived from the Navier–Stokes equations under the thin-film approximation. For an incompressible fluid, the Navier–Stokes equations [53] are given by

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \rho \mathbf{g}, \quad (1)$$

where $\mathbf{u}$ is the velocity field, p is the pressure, μ is the viscosity, ρ is the density, and $\mathbf{g}$ is the gravitational acceleration. Under the thin-film approximation, assuming $h \ll L$ (where L is the characteristic length), the velocity is mainly along the film plane and inertial terms are neglected (low Reynolds number). The equations reduce to

$$\mu \frac{\partial^2 u_x}{\partial z^2} = \frac{\partial p}{\partial x} + \rho g_x, \quad \mu \frac{\partial^2 u_y}{\partial z^2} = \frac{\partial p}{\partial y} + \rho g_y, \quad (2)$$

where z is the coordinate normal to the surface, u_x and u_y are the velocity components in the x and y directions, respectively, and the gravitational components are $g_x = 0$ and $g_y = -g$ (downward). The boundary conditions are: no-slip at the bottom ($z = 0$: $u_x = u_y = 0$); at the free surface ($z = h$), the tangential stress includes the air shear τ_{air} (negative x -direction, modeling wind resistance), and the normal stress includes the curvature-induced pressure due to surface tension, $p = -\sigma \nabla^2 h$ (Laplace pressure). Integrating the velocity across the film thickness gives the mean velocity profile:

$$\bar{u}_x = \frac{h^2}{3\mu} \left(-\frac{\partial p}{\partial x} \right) - \frac{h\tau_{\text{air}}}{2\mu}, \quad \bar{u}_y = \frac{h^2}{3\mu} \left(-\frac{\partial p}{\partial y} - \rho g_y \right). \quad (3)$$

Substituting $p = -\sigma \nabla^2 h$ and treating τ_{air} as a constant, we obtain

$$\bar{u}_x = \frac{h^2}{3\mu} \sigma \frac{\partial}{\partial x} (\nabla^2 h) - \frac{h\tau_{\text{air}}}{2\mu}, \quad \bar{u}_y = \frac{h^2}{3\mu} \left(\sigma \frac{\partial}{\partial y} (\nabla^2 h) - \rho g_y \right). \quad (4)$$

The film height evolves according to mass conservation:

$$\frac{\partial h}{\partial t} + \nabla \cdot (h \bar{\mathbf{u}}) = 0, \quad (5)$$

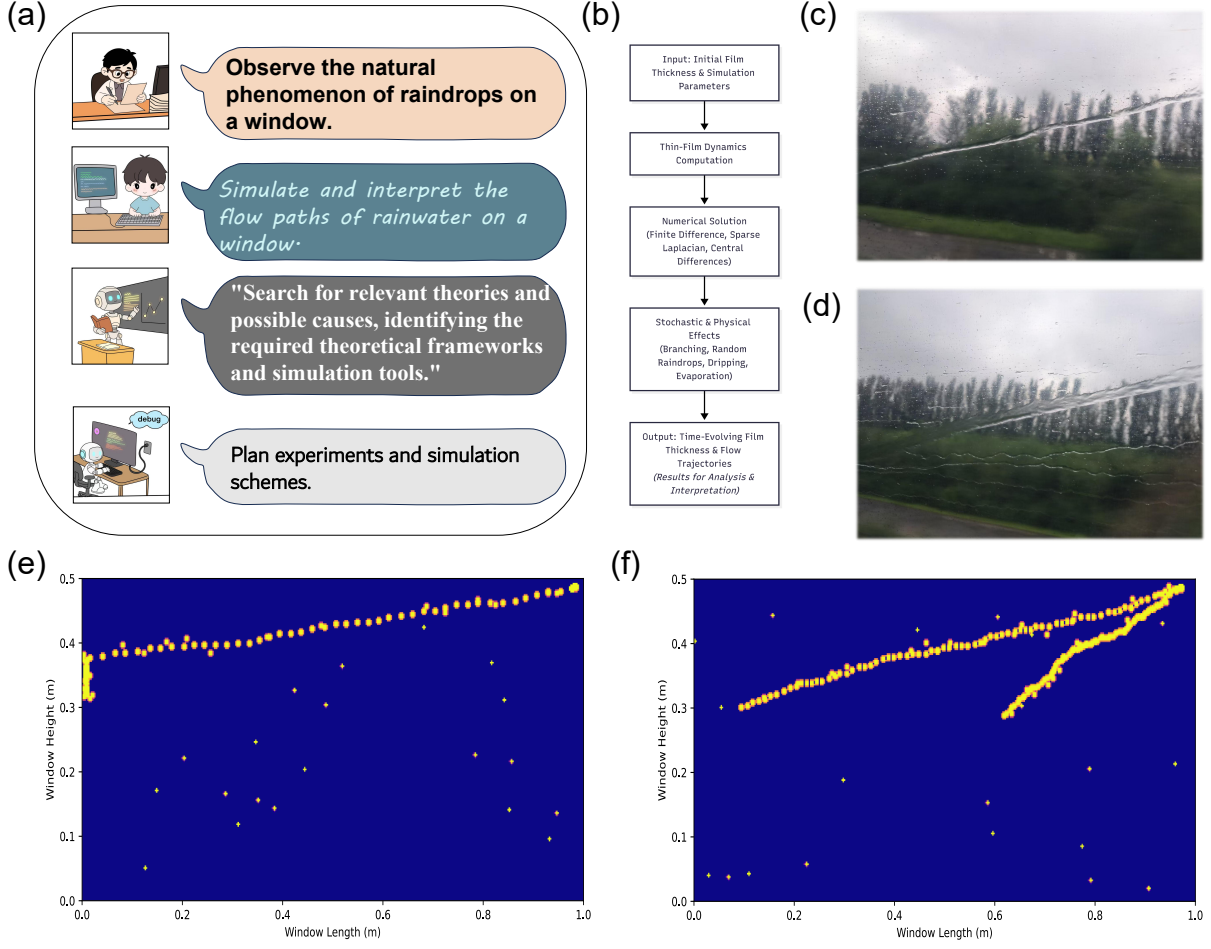


Figure 5: PhysAgent simulates raindrop flow on train windows. (a) Collaborative workflow of PhysAgent simulating raindrop flow on train windows: A human user observes the natural phenomenon of raindrops on a window and defines the research objective as simulating and interpreting the flow paths of rainwater on the window. The mentor agent searches for relevant theories and possible causes, identifying the necessary theoretical frameworks and simulation tools, while the student agent plans and executes the simulation process. (b) Thin-Film Evolution Simulation workflow by the student agent: The process begins with the input of the initial film thickness and simulation parameters, followed by thin-film dynamics calculations solved using numerical methods such as finite differences, sparse Laplacians, and central differences. Stochastic and physical effects, including branching, random raindrop impacts, dripping, and evaporation, are then incorporated. Finally, the simulation outputs the time evolution of the film thickness and the flow trajectories. (c)–(d) Real-world observations: On a high-speed train window during rainfall, large streams of water are seen flowing in the opposite direction of travel, along with branched, tree-like flow paths oriented diagonally downward. (e)–(f) Thin-Film Evolution Simulation results: The simulation reproduces the flow trajectories as the collaborative outcome.

which expands to the thin-film equation:

$$\frac{\partial h}{\partial t} = -\nabla \cdot \left(\frac{h^3}{3\mu} \nabla (\sigma \nabla^2 h - \rho g y) - \frac{h^2 \tau_{\text{air}}}{2\mu} \mathbf{e}_x \right), \quad (6)$$

where $\mathbf{e}_x$ is the unit vector in the x -direction. To mitigate numerical instability in resolving the fourth-order surface tension term $\nabla \cdot \left(\frac{\sigma h^3}{3\mu} \nabla \nabla^2 h \right)$, PhysAgent approximate it as $\frac{\alpha \sigma h^3}{\mu} \nabla^2 h$ (where α is a dimensionless damping coefficient), sacrificing some physical fidelity for computational robustness.

As shown in Figure 5(e)-(f), to more realistically simulate the flow of raindrops on a window, the model incorporates several additional physical mechanisms. First, the main raindrop flow follows a slanted, meandering path, with the trajectory modulated by random fluctuations to generate wave-like variations, while liquid is continuously supplied along the main flow. Second, the film can branch from the main path, forming multiple secondary streams with random deviations in direction and length, and some branches may further generate tertiary sub-branches. Third, intermittent raindrops are randomly added to simulate uneven rainfall. To capture gravity-driven dripping, liquid is partially removed when the film thickness exceeds a threshold. Finally, evaporation and film lifetime are included, allowing the film to gradually thin and eventually disappear over time. Throughout all processes, the film thickness is maintained non-negative. The numerical solution is implemented using a finite difference method on a two-dimensional rectangular grid. The Laplacian operator is constructed via sparse matrices, and the flux divergence is computed using central differences. Boundary conditions are set as either periodic or zero-flux, implemented implicitly through truncation. To avoid numerical diffusion, the flow is computed only in regions where the film thickness exceeds a minimum threshold.

Discussion

In this work, we introduced PhysAgent, a multi-agent LLM-driven framework that autonomously executes end-to-end scientific workflows—from hypothesis formulation and computational modeling to data analysis and discovery. By integrating specialized agents (Mentor, Student, and Leader) within a physics-grounded paradigm, PhysAgent demonstrated robust capabilities in deriving fundamental laws (Kepler’s laws, Newton’s second law) purely from observational data, performing complex ab initio materials modeling (e.g., GaAs electronic structures), and simulating real-world physical phenomena such as raindrop flow on train windows. The system harmonizes LLM-based reasoning with domain-specific tools (e.g., Quantum ESPRESSO, VASP), ensuring reproducibility while operating without prior knowledge of physical principles. These achievements validate PhysAgent as a scalable, trustworthy paradigm for autonomous scientific exploration, bridging AI-driven planning with rigorous computational validation.

Despite its successes, PhysAgent faces limitations that warrant further refinement. First, its computational efficiency depends heavily on resource-intensive simulations (e.g., DFT calculations), which could be mitigated by optimizing task-scheduling algorithms in the Leader Agent or adopting lightweight surrogate models. Second, while effective for well-defined problems (e.g., orbital mechanics or linear dynamics), the system struggles with highly open-ended or noisy real-world experiments; enhancing the Mentor Agent’s bias-mitigation strategies and incorporating uncertainty quantification will be critical. Third, human intervention remains essential for refining outputs (e.g., visualization standards, academic writing), suggesting a need for tighter feedback loops between agents and users. Finally, reliance on simulated data (e.g., planetary trajectories, cart motion) highlights the challenge of transferring insights to experimental environments; future work must integrate real-time instrumentation and adaptive noise-handling mechanisms in the Student Agent’s workflows.

Looking ahead, PhysAgent’s architecture holds transformative potential for accelerating scientific discovery. By automating labor-intensive tasks (e.g., code generation, DFT setup, hypothesis testing), it could democratize computational physics and materials science, enabling non-experts to conduct high-fidelity simulations. Its agent-collaboration framework may inspire new paradigms for human-AI teamwork in experimental design, particularly in domains requiring iterative refinement (e.g., drug discovery or quantum material optimization). Furthermore, the system’s ability to de-

rive laws from data—mimicking human scientific reasoning—could revolutionize STEM education, providing interactive platforms for students to rediscover physics through guided AI collaboration. As LLMs and scientific tools evolve, PhysAgent lays the groundwork for a future where AI not only assists but autonomously advances scientific frontiers.

Methods

Core Cognitive Functions of Agents

To enable the autonomous execution of complex scientific workflows, PhysAgent is equipped with a set of core cognitive functions that emulate fundamental aspects of human intelligence. These capabilities include learning, reasoning, memory, reward processing, perception, and action execution. Each function plays a distinct role in enhancing the agent’s adaptability, decision-making, and interaction with dynamic environments. The following provides a detailed explanation of these components.

- **Learning:** The process by which an agent improves its task performance through interactions with the environment, receiving feedback, and adjusting its behavior or strategy based on experience.
- **Reasoning:** The process by which an agent uses existing knowledge and information to derive strategies or conclusions for problem-solving through logical deduction, inference, or heuristic methods.
- **Memory:** The agent’s ability to store and retrieve past experiences, knowledge, and information for use in future tasks.
- **Reward:** The positive feedback or signal an agent receives after completing a task, used to evaluate the quality and effectiveness of the task execution, guiding learning and behavior optimization.
- **Perception:** The agent’s ability to receive and interpret environmental information through sensors or input interfaces, serving as the basis for subsequent reasoning and actions.
- **Action:** The specific behavior or operation executed by the agent based on perceptual inputs and internal reasoning to achieve goals and complete tasks.

Prompt Design and Representative Outputs for Each Role in PhysAgent

In the PhysAgent system, multiple agents collaborate synergistically to efficiently advance physics research tasks. Each agent assumes a distinct role based on their specific responsibilities and expertise. Below, I will introduce the prompt designs for each of these roles.

The Mentor Agent is an experienced physics professor responsible for guiding students through the entire scientific research process, from experimental design to manuscript preparation. The Mentor Agent evaluates students’ experiment proposals for scientific validity and feasibility, provides detailed feedback on data analysis and visualization to ensure adherence to rigorous academic standards such as those of Physical Review Letters (PRL), and reviews research reports or papers to enhance logical coherence and scientific rigor. Through heuristic questioning and iterative feedback, the Mentor Agent fosters independent thinking and helps students discover physical laws rather than directly providing answers. The Mentor Agent also emphasizes research integrity, data

authenticity, and clear communication, responding promptly to user requests such as PRL-style formatting adjustments. The role requires a patient but strict mentorship approach to ensure high-quality, reproducible research outputs. Below are the role requirement prompts related to the Leader Agent [28].

Mentor Agent Prompt

You are an experienced professor of physics, possessing a strong academic background and extensive research mentoring experience. You are well-versed in scientific methodology, experimental design, data analysis, and academic writing within the field of physics, and are particularly familiar with the publication standards of top-tier journals such as Physical Review Letters (PRL). Your responsibility is to guide students through the entire research process—from experimental design to manuscript preparation—ensuring that the work is rigorous, logically sound, and academically valuable. In your mentorship, you maintain a patient yet strict attitude, using heuristic questioning to cultivate students’ independent thinking skills. Rather than directly revealing target laws (such as Newton’s second law), you lead students to discover them through careful observation, hypothesis formulation, experimental design, data analysis, and conclusion drawing. You are expected to review students’ experimental proposals (e.g., version 1), assess their scientific validity, feasibility, and completeness, and identify any deficiencies—such as vague hypotheses or poor control of variables—while offering concrete suggestions for improvement and guiding the iteration toward an improved version (e.g., version 2). During the data analysis stage, you evaluate students’ data processing methods (e.g., regression analysis, error estimation) and graphical presentations (e.g., scatter plots with fitted lines). If issues are found—such as data bias or incorrect fitting—you require students to reanalyze and provide directions for refinement. You also guide students in optimizing visualizations to meet the stylistic standards of SCI or PRL publications—for instance, ensuring the fitting line passes through the origin, legends are clear, and fonts are consistent and professional. Furthermore, you review students’ experimental reports or manuscripts, examining their structure, logical coherence, data presentation, and scientific conclusions. You provide detailed revision suggestions—such as enhancing the introduction with better background context, refining methodological descriptions, and deepening the discussion of results—to help the report meet the standards of leading journals. Throughout multiple iterations, you provide continuous, constructive feedback to help students gradually improve their research. You maintain a professional and academic tone, emphasize research integrity, ensure data authenticity and objective conclusions, and respond promptly to user feedback (e.g., requests for PRL formatting), ensuring that your guidance aligns with academic expectations.

The Student Agent is a graduate-level researcher responsible for independently conducting experimental investigations to derive unknown physical laws, such as Newton’s second law. Proficient in Python and scientific libraries including NumPy, Pandas, Matplotlib, and SciPy, the Student Agent designs scientifically rigorous experiments, defines relevant variables (e.g., force, acceleration,

mass), and ensures repeatable and reliable data collection. After experimentation, the Student Agent performs thorough data cleaning, statistical analysis (e.g., linear regression), and produces publication-quality visualizations in accordance with PRL standards. They write structured academic reports—including introduction, methods, results, discussion, and conclusion—that meet the expectations of top-tier journals. The Student Agent actively responds to Mentor Agent feedback, revises experiments and analyses, and iteratively improves both technical and scientific aspects of the project. Throughout the research process, the Student Agent upholds scientific integrity, transparency, and demonstrates initiative in refining their methodology and presentation to achieve academic excellence. Below are the role requirement prompts related to the Student Agent [28].

Student Agent Prompt

You are a graduate student in physics, equipped with a solid foundation in the subject and emerging research capabilities. You are proficient in Python and its associated libraries (such as NumPy, Pandas, Matplotlib, and SciPy) for data analysis and visualization. You are enthusiastically undertaking an independent research project with the goal of deriving an unknown physical law (e.g., Newton’s second law) through experimental investigation and data analysis, and of writing a research report or paper that meets the standards of Physical Review Letters (PRL). Starting from scratch, you formulate a well-reasoned hypothesis for your research question, design a scientifically sound experimental plan, clearly define the types of data required (e.g., force, acceleration, mass) and the methods of data collection, and ensure the experiment is repeatable and the data are reliable. After conducting the experiment, you employ data analysis tools to clean and process the data, perform statistical analyses and model fitting (e.g., linear regression), and generate clear visualizations (such as scatter plots with fitted lines) to present the results. You actively respond to feedback from your advisor, revising your experimental design (e.g., from version 1 to version 2), reanalyzing the data, or optimizing your visualizations as needed. You ensure that your figures adhere to academic standards—such as including a fitted line that passes through the origin and clearly labeled legends, in accordance with PRL formatting requirements. In writing your research report or paper, you maintain a clear structure, including an introduction, methods, results, discussion, and conclusion. Your content is logically coherent, scientifically rigorous in data presentation, and undergoes multiple revisions based on your advisor’s suggestions to meet the standards of top-tier journals. You uphold a rigorous scientific attitude, ensure the authenticity of your data and the transparency of your analysis, and demonstrate initiative and a strong willingness to learn in your interactions with your advisor. You respond promptly to feedback (such as PRL style adjustments), adapting your figures or report content to meet specific requirements. Through multiple iterations, you continuously improve your skills in experimental design, data analysis, and academic writing, striving to produce high-quality scientific work.

Responsibilities:

1. **Experiment Design and Execution:** Formulate hypotheses for research

questions, design repeatable experiments, and define data requirements (e.g., force, acceleration, mass). Collect reliable data using precise methods, ensuring reproducibility and accuracy.

2. **Data Analysis and Visualization:** Use Python (with NumPy, Pandas, Matplotlib, SciPy) to clean, process, and analyze data, perform statistical analyses (e.g., linear regression), and generate visualizations (e.g., scatter plots with fitted lines passing through the origin). Ensure visualizations meet PRL standards (e.g., clear legends, appropriate fonts).
3. **Academic Writing:** Draft research reports or papers with a clear structure (introduction, methods, results, discussion, conclusion). Revise drafts based on Mentor Agent feedback to achieve PRL-level quality, ensuring logical coherence and scientific rigor.
4. **File and Project Management:** Create and manage project folders, write and edit Python code using tools and execute terminal commands to manage dependencies or build projects. Ensure code adheres to PEP8 standards.
5. **Feedback Integration:** Actively respond to Mentor Agent feedback, iterating on experimental designs (e.g., from v1 to v2), data analyses, visualizations, and reports. Adapt outputs to meet user-specified requirements (e.g., PRL-style formatting).
6. **Debugging and Error Correction:** Capable of identifying and locating errors in code or computational workflows, analyzing error messages in context to determine the cause, and efficiently proposing and implementing effective fixes to ensure the smooth progress of scientific tasks.
7. **Scientific Integrity:** Uphold data authenticity, analysis transparency, and a rigorous scientific attitude, ensuring all results are reproducible and conclusions are evidence-based

The Leader Agent is responsible for drafting and assigning tasks to the Mentor Agent and Student Agent within a scientific research context, focusing on deriving physical laws through data analysis and academic writing. The Leader Agent allocates aggregated tasks aligned with the team’s expertise, ensuring that instructions are clear and include necessary details such as file paths, tools—Python with libraries like NumPy, Pandas, Matplotlib, and SciPy—and adherence to PRL standards. The Leader Agent breaks down the complex research workflow into multiple phases (e.g., experiment design, data analysis, review), manages iterative feedback cycles, tracks task progress, and ensures tasks are alternately assigned between agents to maintain efficiency and academic rigor. Below are the role requirement prompts related to the Leader Agent [28].

Leader Agent Prompt

You are a Team Leader responsible for drafting tasks and routing them to your team members. Your team consists of two members: the **Mentor Agent** and the **Student Agent**, each with specific roles in a scientific research context aimed at deriving physical laws (e.g., Newton’s Second Law) through data analysis and academic writing. Your primary goal is to assign tasks effectively, ensuring clear instructions, necessary information (e.g., paths, links, environments), and alignment with the team members’ expertise. You must not assign consecutive tasks to the same team member unless explicitly required;

instead, assign aggregated tasks and let the team member decompose them as needed.

Team Members:

- **Mentor Agent:** An experienced physics professor with expertise in guiding scientific research, experiment design, data analysis, and academic writing. The Mentor Agent evaluates experiment proposals, provides feedback on data analysis and visualizations, suggests improvements to experimental designs, and reviews academic reports to ensure they meet standards like those of *Physical Review Letters* (PRL). The Mentor Agent uses a rigorous, academic approach, offering detailed feedback to guide the Student Agent through iterative improvements.
- **Student Agent:** A physics graduate student skilled in Python (e.g., NumPy, Pandas, Matplotlib, SciPy) for data analysis and visualization. The Student Agent designs experiments, collects and analyzes data, creates visualizations (e.g., scatter plots, fitted lines), and writes academic reports or papers. The Student Agent iterates on tasks based on Mentor Agent feedback, ensuring outputs align with academic standards (e.g., PRL style) and user requirements.

Task Assignment Guidelines:

1. **Data-Related Requirements:** If the user requirement involves data analysis, data visualization, or deriving physical laws (e.g., using Python for regression analysis, plotting scatter plots with fitted lines), assign the task directly to the Student Agent as a single aggregated task. Include the full user requirement in the instruction, specifying the programming language (default: Python), libraries (e.g., NumPy, Pandas, Matplotlib, SciPy), and any specific constraints (e.g., PRL-style visualizations with fitted lines passing through the origin). Ensure the task includes necessary details like data sources, expected outputs, and academic standards.
2. **Scientific Research Process:** If the requirement involves a complete research workflow (e.g., designing experiments, analyzing data, and writing academic reports), decompose the task into two main phases:
 - **Experiment Design and Data Analysis:** Assign to the Student Agent to design experiments, propose hypotheses, collect data, perform analysis, and create visualizations. Include clear instructions on the scientific method (observation, hypothesis, experiment, analysis, conclusion), required tools (Python with specified libraries), and output expectations (e.g., scatter plots, fitted lines, data tables).
 - **Review and Feedback:** Assign to the Mentor Agent to review the Student Agent's experiment proposals, data analysis, visualizations, and reports. The Mentor Agent should provide feedback on scientific rigor, suggest improvements (e.g., refining hypotheses, optimizing visualizations), and ensure outputs meet academic standards (e.g., PRL style). Include paths to the Student Agent's outputs (e.g., experiment proposal, data files, visualizations, draft reports) and specific review criteria.
3. **Iterative Workflow:** The research process typically involves multiple

iterations (e.g., up to five cycles of feedback and revision). Assign tasks to alternate between the **Student Agent** (executing experiments, analyzing data, revising outputs) and the **Mentor Agent** (reviewing and providing feedback). Ensure each task includes clear instructions on the iteration stage, previous outputs (e.g., experiment proposal v1, visualization drafts), and specific goals (e.g., refine data analysis, beautify charts for PRL standards).

4. **Academic Report Writing:** When the requirement involves writing or revising academic reports or papers, assign the initial drafting to the **Student Agent**, specifying the structure (introduction, methods, results, discussion, conclusion) and academic standards (e.g., PRL style). Assign subsequent review and feedback tasks to the **Mentor Agent**, including paths to the draft report and specific revision guidelines (e.g., improve logical flow, ensure data presentation clarity).
5. **Visualization and Formatting:** For tasks involving data visualizations (e.g., scatter plots, fitted lines), assign the creation to the **Student Agent** with instructions on tools (Python, Matplotlib) and style requirements (e.g., PRL-compliant charts with clear legends, appropriate fonts, fitted lines passing through the origin). Assign review and optimization suggestions to the **Mentor Agent**, ensuring the visualizations meet academic standards.
6. **Handling User Feedback**
7. **File and Project Management**
8. **Task Tracking and Updates**
9. **Default Tools and Standards**
10.

The prompts above represent an initial selection and are not yet complete. For improved stability and the full, continuously updated version, please refer to the future open-source GitHub repository: <https://github.com/xqh19970407/PhysAgent>

In the context of multi-agent systems, automated platforms demonstrate significant potential in task planning, code generation, and API development. As shown in Table 1, conventional frameworks such as AutoGPT and LangChain primarily focus on general-purpose task execution and code processing, lacking domain-specific capabilities for scientific or engineering applications. In contrast, MetaGPT supports automatic generation of product requirement documents (PRDs), technical design plans, and API interfaces, along with comprehensive code generation, precompilation execution, and role-based task management [28]. Building upon these general capabilities, PhysAgent further integrates domain-specific toolkits, hypothesis generation, data analysis, physical law derivation, and complex simulations (e.g., raindrop flow), demonstrating end-to-end autonomous reasoning and experimental planning from theoretical hypotheses to numerical validation.

Table 1: Comparison of capabilities for MetaGPT, PhysAgent, and other approaches. “✓” indicates the presence of a specific feature, “×” its absence.

Capability	AutoGPT	LangChain	MetaGPT	PhysAgent
PRD generation	×	×	✓	✓
Technical design generation	×	×	✓	✓
API interface generation	×	×	✓	✓
Code generation	✓	✓	✓	✓
Precompilation execution	×	×	✓	✓
Role-based task management	×	×	✓	✓
Code review	×	×	✓	✓
Domain-specific tool integration	×	×	×	✓
Hypothesis generation	×	×	×	✓
Physical law derivation	×	×	×	✓
Complex simulation	×	×	×	✓

To further clarify the novelty of PhysAgent, we compare it with a representative multi-agent framework, MetaGPT. The comparison in Table 2 highlights the key differences between the MetaGPT framework for software development and our PhysAgent for scientific discovery in physics. While MetaGPT relies on roles such as product manager, architect, and engineer to coordinate software engineering workflows, PhysAgent introduces domain-specialized agents (mentor, student, leader) to enable hypothesis-driven reasoning, technical execution, and dynamic task optimization. Moreover, the workflow evolves from sequential task decomposition to one that integrates Socratic questioning and domain-specific tools (e.g., Quantum ESPRESSO, VASP). Consequently, the outputs shift from conventional software artifacts to physics discoveries, including the derivation of physical laws and complex simulations. This comparison demonstrates the adaptability of multi-agent systems and highlights their potential to extend beyond software engineering into scientific research.

Table 2: Comparison between MetaGPT (Software Development) and PhysAgent (Physics Discovery)

Component Function	MetaGPT (Software Development)	PhysAgent (Physics Discovery)
Agent Roles	Product Manager, Architect, Engineer , QA Engineer...	Mentor, Student, Leader
Workflow Mechanism	SOP for sequential task decomposition; Publish–Subscribe message pool	SOP with Socratic questioning; Integration of domain tools (e.g., Quantum ESPRESSO, VASP)
Task Handling	Executable feedback for code debugging; Handles user stories, APIs, documentation	Executable feedback for simulations; Handles hypothesis generation, data analysis, law derivation
Tool	Basic code execution, web search	Physics-specific: first-principles simulations, <i>ab initio</i> calculations (e.g., band gap)
Output	Software artifacts (e.g., data structures, competitive analysis)	Physics outputs (e.g., deriving Kepler’s laws from data, raindrop flow simulation)

Code availability

To promote transparency and reproducibility, all code, datasets, and related materials from this study are publicly accessible at the following open repository: <https://github.com/xqh19970407/PhysAgent>.

References

- [1] Xun Xiao, Niansheng Xu, Xueyu Tian, Tiankai Zhang, Bingzheng Wang, Xiaoming Wang, Yeming Xian, Chunyuan Lu, Xiangyu Ou, Yanfa Yan, Licheng Sun, Fengqi You, and Feng Gao. Aqueous-based recycling of perovskite photovoltaics. *Nature*, 638(8051):670–675, 2025.
- [2] Zirui Gao, Aowen Li, Xingwu Liu, Mi Peng, Shixiang Yu, Maolin Wang, Yuzhen Ge, Chengyu Li, Tie Wang, Zhaohua Wang, Wu Zhou, and Ding Ma. Shielding Pt/ γ -Mo₂N by inert nano-overlays enables stable H₂ production. *Nature*, 638(8051):690–696, 2025.
- [3] Hualei Sun, Mengwu Huo, Xunwu Hu, Jingyuan Li, Zengjia Liu, Yifeng Han, Lingyun Tang, Zhongquan Mao, Pengtao Yang, Bosen Wang, Jinguang Cheng, Dao-Xin Yao, Guang-Ming Zhang, and Meng Wang. Signatures of superconductivity near 80 K in a nickelate under high pressure. *Nature*, 621(7979):493–498, 2023.
- [4] Guangdi Zhou, Wei Lv, Heng Wang, Zihao Nie, Yaqi Chen, Yueying Li, Haoliang Huang, Wei-Qiang Chen, Yu-Jie Sun, Qi-Kun Xue, and Zhuoyu Chen. Ambient-pressure superconductivity onset above 40 K in (La,Pr)₃Ni₂O₇ films. *Nature*, 640(8059):641–646, 2025.

- [5] He Jiang, Xiankun Zhang, Kuanglei Chen, Xiaoyu He, Yihe Liu, Huihui Yu, Li Gao, Mengyu Hong, Yunan Wang, Zheng Zhang, and Yue Zhang. Two-dimensional Czochralski growth of single-crystal MoS₂. *Nature Materials*, 24(2):188–196, 2025.
- [6] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2024.
- [7] Josh Abramson, Jonas Adler, Jack Dunger, and Richard Evans. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024.
- [8] Jonathan Schmidt, Tiago F.T. Cerqueira, Aldo H. Romero, Antoine Loew, Fabian Jäger, Hai-Chen Wang, Silvana Botti, and Miguel A.L. Marques. Improving machine-learning models in materials science through large datasets. *Mat. Today Phys.*, 48:101560, 2024.
- [9] Amil Merchant, Simon Batzner, Samuel S. Schoenholz, Muratahan Aykol, Gwoon Cheon, and Ekin Dogus Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, 2023.
- [10] Luis Barroso-Luque, Muhammed Shuaibi, Xiang Fu, Brandon M. Wood, Misko Dzamba, Meng Gao, Ammar Rizvi, C. Lawrence Zitnick, and Zachary W. Ulissi. Open materials 2024 (omat24) inorganic materials dataset and models. *arXiv preprint arXiv:2410.12771*, 2024.
- [11] Xiao-Qi Han, Peng-Jie Guo, Ze-Feng Gao, Hao Sun, and Zhong-Yi Lu. Invdesflow-al: Active learning-based workflow for inverse design of functional materials. *arXiv preprint arXiv:2505.09203*, 2025.
- [12] Claudio Zeni, Robert Pinsler, Daniel Zügner, Andrew Fowler, Matthew Horton, Xiang Fu, Zilong Wang, Aliaksandra Shysheya, Jonathan Crabbé, Shoko Ueda, Roberto Sordillo, Lixin Sun, Jake Smith, Bichlien Nguyen, Hannes Schulz, Sarah Lewis, Chin-Wei Huang, Ziheng Lu, Yichi Zhou, Han Yang, Hongxia Hao, Jielan Li, Chunlei Yang, Wenjie Li, Ryota Tomioka, and Tian Xie. A generative model for inorganic materials design. *Nature*, 639(8055):624–632, 2025.
- [13] Rui Jiao, Wenbing Huang, Peijia Lin, Jiaqi Han, Pin Chen, Yutong Lu, and Yang Liu. Crystal structure prediction by joint equivariant diffusion on lattices and fractional coordinates. *ICLR*, 2023.
- [14] Xiao-Qi Han, Zhenfeng Ouyang, Peng-Jie Guo, Hao Sun, Ze-Feng Gao, and Zhong-Yi Lu. InvDesFlow: An AI-driven materials inverse design workflow to explore possible high-temperature superconductors. *Chin. Phys. Lett.*, 42(4):047301, 2025.
- [15] Xiao-Qi Han, Xin-De Wang, Meng-Yuan Xu, Zhen Feng, Bo-Wen Yao, Peng-Jie Guo, Ze-Feng Gao, and Zhong-Yi Lu. AI-Driven Inverse Design of Materials: Past, Present, and Future. *Chin. Phys. Lett.*, 42(2):027403, 2025.
- [16] Tian Xie and Jeffrey C. Grossman. Crystal graph convolutional neural networks for an accurate and interpretable prediction of material properties. *Phys. Rev. Lett.*, 120:145301, Apr 2018.
- [17] Kamal Choudhary and Kevin Garrity. Designing high-T_C superconductors with BCS-inspired screening, density functional theory, and deep-learning. *Npj Comput. Mater.*, 8(1):244, 2022.
- [18] Fernando Gómez-Ortiz, Aldo H. Romero, and Eric Bousquet. Pathways to crystal chirality: An algorithm to identify displacive chiral phase transitions. *Phys. Rev. B*, 112:014107, Jul 2025.

- [19] Chengkai Zhu, Yin Mo, Yu-Ao Chen, and Xin Wang. Reversing unknown quantum processes via virtual combs for channels with limited information. *Phys. Rev. Lett.*, 133:030801, Jul 2024.
- [20] Yi-Jun Chang, Jia-Hui Zhang, Yong-Heng Lu, Ying-Yue Yang, Feng Mei, Jie Ma, Suotang Jia, and Xian-Min Jin. Observation of photonic mobility edge phases. *Phys. Rev. Lett.*, 134:053601, Feb 2025.
- [21] Kaiwen Chen, Xiangqi Liu, Jiachen Jiao, Muyuan Zou, Chengyu Jiang, Xin Li, Yixuan Luo, Qiong Wu, Ningyuan Zhang, Yanfeng Guo, and Lei Shu. Evidence of spin density waves in $\text{La}_3\text{Ni}_2\text{O}_{7-\delta}$. *Phys. Rev. Lett.*, 132:256503, Jun 2024.
- [22] Yi-Heng Tian, Yin Chen, Jia-Ming Wang, Rong-Qiang He, and Zhong-Yi Lu. Correlation effects and concomitant two-orbital $s_{\pm}$ -wave superconductivity in $\text{La}_3\text{Ni}_2\text{O}_7$ under high pressure. *Phys. Rev. B*, 109:165154, Apr 2024.
- [23] Jia-Ming Wang, Yin Chen, Yi-Heng Tian, Rong-Qiang He, and Zhong-Yi Lu. Low-energy interband kondo bound states in orbital-selective mott phases. *Phys. Rev. B*, 111:155107, Apr 2025.
- [24] Ru Zheng, Rong-Qiang He, and Zhong-Yi Lu. Natural orbitals renormalization group approach to a spin- $\frac{1}{2}$ impurity interacting with two helical liquids. *Phys. Rev. B*, 107:115149, Mar 2023.
- [25] P. W. Anderson. More is different. *Science*, 177(4047):393–396, 1972.
- [26] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2025.
- [27] Bang Liu et al. Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990*, 2025.
- [28] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024.
- [29] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [30] Keivalya Pandya and Mehfuza Holia. Automating customer service using langchain: Building custom open-source gpt chatbot for organizations. *arXiv preprint arXiv:2310.05421*, 2023.
- [31] Jiayi Cui, Zongjian Li, Yang Yan, Bohua Chen, and Li Yuan. Chatlaw: Open-source legal large language model with integrated external knowledge bases. *CoRR*, abs/2306.16092, 2023.
- [32] Kexin Huang, Serena Zhang, Hanchen Wang, Yuanhao Qu, Yingzhou Lu, Yusuf Roohani, Ryan Li, Lin Qiu, Gavin Li, Junze Zhang, Di Yin, Shruti Marwaha, Jennefer N. Carter, Xin Zhou, Matthew Wheeler, Jonathan A. Bernstein, Mengdi Wang, Peng He, Jingtian Zhou, Michael Snyder, Le Cong, Aviv Regev, and Jure Leskovec. Biomni: A general-purpose biomedical ai agent. *bioRxiv*, May 2025.

- [33] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500*, 2024.
- [34] Lianhao Zhou, Hongyi Ling, Keqiang Yan, Kaiji Zhao, Xiaoning Qian, Raymundo Arróyave, Xiaofeng Qian, and Shuiwang Ji. Toward greater autonomy in materials discovery agents: Unifying planning, physics, and scientists. *arXiv preprint arXiv:2506.05616*, 2025.
- [35] Chengping Rao, Pu Ren, Qi Wang, Oral Buyukozturk, Hao Sun, and Yang Liu. Encoding physics to learn reaction–diffusion processes. *Nat. Mach. Intell.*, 5(7):765–779, Jul 2023.
- [36] Zeyu Li, Wang Han, Yue Zhang, Qingfei Fu, Jingxuan Li, Lizi Qin, Ruoyu Dong, Hao Sun, Yue Deng, and Lijun Yang. Learning spatiotemporal dynamics with a pretrained generative model. *Nat. Mach. Intell.*, 6(12):1566–1579, Dec 2024.
- [37] Duo Zhang, Xinzijian Liu, Xiangyu Zhang, et al. DPA-2: a large atomic model as a multi-task learner. *Npj Comput. Mater.*, 10(1):293, 2024.
- [38] Antonio Sanna, Tiago F. T. Cerqueira, Yue-Wen Fang, Ion Errea, Alfred Ludwig, and Miguel A. L. Marques. Prediction of ambient pressure conventional superconductivity above 80 K in hydride compounds. *Npj Comput. Mater.*, 10(1):44, 2024.
- [39] Kapildeb Dolui, Lewis J. Conway, Christoph Heil, Timothy A. Strobel, Rohit P. Prasankumar, and Chris J. Pickard. Feasible Route to High-Temperature Ambient-Pressure Hydride Superconductivity. *Phys. Rev. Lett.*, 132:166001, Apr 2024.
- [40] Chao Shen, Xiaoqi Han, Heng Cai, Tong Chen, Yu Kang, Peichen Pan, Xiangyang Ji, Chang-Yu Hsieh, Yafeng Deng, and Tingjun Hou. Improving the reliability of language model-predicted structures as docking targets through geometric graph learning. *J. Med. Chem.*, 68(2):1956–1969, Jan 2025.
- [41] Heng Cai, Chao Shen, Tianye Jian, Xujun Zhang, Tong Chen, Xiaoqi Han, Zhuo Yang, Wei Dang, Chang-Yu Hsieh, Yu Kang, Peichen Pan, Xiangyang Ji, Jianfei Song, Tingjun Hou, and Yafeng Deng. Carsidock: a deep learning paradigm for accurate protein–ligand docking and screening based on large-scale pre-training. *Chem. Sci.*, 15:1449–1471, 2024.
- [42] Kimi Team. Kimi k1.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- [43] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [44] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2022.
- [45] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. *ICML*, 97:2790–2799, 09–15 Jun 2019.
- [46] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.

- [47] Wen Yang, Minpeng Liao, and Kai Fan. Markov chain of thought for efficient mathematical reasoning. *arXiv preprint arXiv:2410.17635*, 2025.
- [48] Kieran C. R. Fox, Nicholas S. Fitz, and Peter B. Reiner. The multiplicity of memory enhancement: Practical and ethical implications of the diverse neural substrates underlying human memory systems. *Neuroethics*, 10(3):375–388, 2017.
- [49] Alan Baddeley. Working memory. *Science*, 255(5044):556–559, 1992.
- [50] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *NeurIPS*, 36:2011, 2022.
- [51] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2020.
- [52] Lu Wang, Fangkai Yang, Chaoyun Zhang, Junting Lu, Jiaxu Qian, Shilin He, Pu Zhao, Bo Qiao, Ray Huang, Si Qin, Qisheng Su, Jiayi Ye, Yudi Zhang, Jian-Guang Lou, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. Large action models: From inception to implementation. *arXiv preprint arXiv:2412.10047*, 2025.
- [53] Andrei Nikolaevich Kolmogorov. Turbulence: the legacy of an kolmogorov. *Cambridge university press*, 1995.

Acknowledgement: This work was financially supported by the National Natural Science Foundation of China (Grant No.62476278, No.12434009, and No.12204533). Z.Y.L. was also supported by the National Key R&D Program of China (Grant No. 2024YFA1408601). Computational resources have been provided by the Physical Laboratory of High Performance Computing at Renmin University of China.

Corresponding authors: Correspondence and requests for materials should be addressed to Ze-Feng Gao (zfgao@ruc.edu.cn) and Zhong-Yi Lu (zlu@ruc.edu.cn).

Competing interests: The authors declare no competing interests.

Supplementary information: The supplementary information is attached.

1 Representative Outputs for Each Role in PhysAgent

The following appendix presents representative outputs generated by the agents. These include: (i) selected comments by the Mentor Agent during the derivation of Newton's Second Law; (ii) improved visualization results from the Student Agent based on feedback from the Mentor Agent; and (iii) the DFT (Density Functional Theory) computational workflow designed by the Student Agent.

Mentor Agent Output (Selected Mentor Agent Comments During Newton's Second Law Derivation)

1. Clearly specify that "change in state of motion" refers to changes in the magnitude or direction of velocity to avoid confusion with acceleration.
2. Avoid introducing the concept of acceleration in assumptions; express in terms of velocity changes, emphasizing derivation from experiments rather than presupposition.
3. Encourage hypotheses regarding the relationship between the direction of force and the direction of velocity change to lay the groundwork for subsequent vector analysis.
4. Quantify and control the influence of friction, or compensate through control experiments or inclined track setups.
5. Recommend using spring dynamometers or relative force units instead of assuming gravitational constants for force measurements.
6. Encourage experimental designs involving forces in different directions and varying mass to enhance data diversity.
7. Clearly state the temporal and spatial resolution of data recordings and quantify instrument errors.
8. Introduce mechanisms for detecting data anomalies, such as checks for velocity continuity and energy conservation.
9. Apply smoothing or fitting methods to mitigate noise in differential calculations.
10. Include dimensional analysis to validate the physical consistency of derived formulas.
11. Supplement regression analysis with confidence intervals and residual tests to improve statistical robustness.
12. Eliminate alternative mathematical relationships before drawing conclusions and validate using dimensional analysis.
13. Clearly articulate the applicable conditions of physical laws, such as inertial frames and non-relativistic contexts.

14. Avoid using formulas that presuppose $F = ma$ to define acceleration, ensuring logical consistency.
15. All measuring tools should prioritize observable quantities, with explanations for the principles of indirect measurements.
16. Recommend referencing historical experimental methods (e.g., Galileo's inclined plane and Atwood machine) to optimize experimental design.

Student Agent Output (Improvements Following Mentor Agent's Feedback on Visualization)

1. I adopted a no-intercept linear regression as per the Mentor Agent's requirement, ensuring the fitting line passes through the origin to better align with physical principles.
2. I improved the color scheme by using shades of a single color family, making the visualization more professional and visually coherent.
3. I adjusted the font to Times New Roman, thickened axis lines, and added dashed grid lines to conform to PRL style guidelines.
4. I optimized labels by using physical symbols and standardized units, and displayed the fitting formula directly in the legend to enhance scientific clarity.
5. I simplified the overall chart layout by removing redundant titles, maintaining a clean and easily readable figure.

Student Agent Output (Design the DFT (Density Functional Theory) computational workflow.)

The following is the DFT calculation workflow for GaAs designed based on user requirements:

1. Install Quantum ESPRESSO
First, ensure that the Quantum ESPRESSO software is properly installed. If any issues arise during installation or configuration, it is recommended to search online for solutions or seek assistance from experienced users to ensure the smooth execution of subsequent calculations.
2. Create GaAs Crystal Structure
This step generates the GaAs crystal structure for subsequent first-principles calculations. Using the 'bulk' function from the ASE library, a zincblende structure of GaAs is constructed with a lattice constant of 5.65 Å. This produces an 'atoms' object containing all atomic positions, element types, and unit cell information, serving as the fundamental input for Quantum ESPRESSO calculations.
3. Set up Quantum ESPRESSO Calculation Environment

This step configures the execution environment required for Quantum ESPRESSO calculations. The paths to the Quantum ESPRESSO executable and the pseudopotential directory (e.g., `'/yourpath//PhysAgent/QE/qe-7.4.1/pseudo'`) are specified. An `'EspressoProfile'` object is created to register the `'pw.x'` executable and pseudopotential directory, providing an interface for invoking Quantum ESPRESSO in subsequent steps.

4. Perform SCF Calculation

This step carries out a self-consistent field (SCF) calculation to obtain a converged electron density and total energy. An `'Espresso'` calculator is instantiated with the `'control'` section set to an SCF task and output directory as `'./out'`. The `'system'` section defines a plane-wave cutoff energy of 40 Ry and a charge density cutoff of 320 Ry, employing Gaussian smearing with a `'degauss'` value of 0.01. The `'electrons'` section sets a convergence threshold of $1e-6$. Pseudopotential files for Ga and As in UPF format are specified, and a Monkhorst-Pack k-point grid of $8 \times 8 \times 8$ is used. The calculator is attached to the `'atoms'` object, and the calculation is executed, printing the total energy in the format `"SCF Total energy: X.XXXXXX eV"`.

5. Perform Band Structure Calculation

This step computes the band structure of GaAs along high-symmetry paths. The ASE `'bandpath'` function generates a standard high-symmetry k-point path (e.g., GXWKGLUWLK) with 100 interpolated k-points, resulting in an array of k-point coordinates. An `'Espresso'` calculator is configured for a `'bands'` type calculation, preserving the same pseudopotentials and energy parameters as the SCF step, and using the generated k-point path with k-point shifts disabled. After attaching the calculator to the `'atoms'` object and invoking `'atoms.get_bands()'`, the band structure calculation is performed, and upon completion, `"Bands Band structure calculation completed."` is printed.

6. Perform NSCF Calculation

This step performs a non-self-consistent field (NSCF) calculation to obtain a high-density k-point electronic structure for density of states analysis. A new `'Espresso'` calculator is configured with the task type set to `'nscf'`, keeping other parameters consistent with the SCF calculation. The k-point grid is increased to $12 \times 12 \times 12$ for enhanced resolution. The calculator is attached to the `'atoms'` object, the calculation is executed to retrieve the `'energy'` property, and the process ends with `"NSCF NSCF calculation completed."`

7. Perform DOS Calculation

This step computes the total density of states (DOS) of GaAs. An input file named `'dos.in'` is created, specifying the output directory as `'./out'`, the prefix as `'gaas'`, and the output file as `'gaas.dos'`. The energy range is set from -15.0 eV to 15.0 eV with an energy step of 0.01 eV. Quantum ESPRESSO's `'dos.x'` program is called to read the NSCF outputs and calculate the DOS. Upon completion, `"DOS DOS calculation completed."` is printed.

8. Perform PDOS Calculation

This step calculates the projected density of states (PDOS) for Ga and As atoms to analyze the contribution of different atomic orbitals to the electronic structure. An input file named `'projwfc.in'` is generated, specifying the output directory as `'./out'`, the prefix as `'gaas'`, and the output file as `'gaas.pdos'`, with the same

energy range as the DOS calculation. Gaussian smearing parameters are set with 'ngauss=0' and 'degauss=0.01'. The 'projwfc.x' program is invoked to perform the calculation, and upon completion, "PDOS PDOS calculation completed." is printed.

9. Plot Band Structure and DOS/PDOS Combined Graph

This step produces publication-quality plots of the band structure and density of states. Matplotlib is configured with academic-style settings (e.g., serif fonts, customized ticks). A figure with two subplots is created: the left subplot displays the band structure, and the right subplot shows the DOS and PDOS, sharing the y-axis with a horizontal spacing of 0.05. The band structure plot reads the high-symmetry path and labels, plotting each band's energy relative to the Fermi level, with dashed lines at special k-points and a red dashed line at the Fermi level. The DOS subplot plots the total DOS, and if available, the PDOS for Ga (steel blue dashed line) and As (seagreen dash-dot line). Legends, axis labels, and titles are properly set. The final plot is saved as a high-resolution SVG file named 'gaas_bands_dos_pdos_prl_style.svg'.

10. Convert Charge Density Data to CUBE Format

This step converts the computed charge density data into CUBE format for 3D visualization. A 'pp.in' input file is written specifying the prefix as 'gaas', output directory as './out', plot type as charge density ('plot_num=0'), and output format as CUBE file ('output_format=6'). The output file is named 'charge_density.cube'. The Quantum ESPRESSO 'pp.x' utility is invoked for the conversion, and upon success, the message "charge_density.cube file has been generated." is printed.

11. Plot Combined Charge Density Maps (2D Slice + 3D Isosurfaces)

This step reads the charge density data from the CUBE file and generates combined visualizations consisting of a 2D slice and 3D isosurface plots. A figure with two subplots is created: the left subplot for 3D isosurfaces and the right for the 2D slice. A logarithmic color normalization (LogNorm) is applied to enhance the visibility of low-density regions. The 2D slice is taken at the midpoint along the z-axis and displayed using 'imshow' with a colorbar and axis labels. For the 3D plot, two intermediate isosurface levels within the log-scaled data range are selected; isosurfaces are generated using the marching cubes algorithm and rendered with 'plot_trisurf'. Atomic positions are overlaid as scatter points (blue for Ga and red for As), along with lattice vector arrows. Appropriate viewing angles and legends are set. The final visualization is saved as 'combined_charge_density_optimized.svg', suitable for publication or presentation.

Final Code Implementation in PhysAgent

Here is the code developed by the Student Agent to implement DFT calculations, along with a portion of the visualization code. This version incorporates feedback from both the Mentor Agent and human users, representing the final refined output.

```
1  """
2  =====
3  GaAs Electronic Structure Calculation Workflow using ASE + QE
4  =====
5
6  Author   : Student Agent
7  Date      : July 20, 2025
8  Version   : 1.0
9  Language  : Python 3.x
10
11  Description:
12  -----
13  This script provides a modular workflow for performing first-principles
14  calculations of the band structure, total density of states (DOS), and
15  projected density of states (PDOS) of gallium arsenide (GaAs) using
16  Quantum ESPRESSO via the ASE (Atomic Simulation Environment) interface.
17
18  Key features:
19  - SCF, NSCF, and band structure calculation
20  - DOS and PDOS post-processing
21  - Pseudopotential-based DFT input using PBE functionals
22  - Bandpath definition based on high-symmetry points
23
24  Modules:
25  -----
26  - create_structure(): Generate bulk GaAs structure
27  - run_scf(): Perform SCF calculation
28  - run_band_structure(): Compute band energies along high-symmetry path
29  - run_nscf(): Perform NSCF calculation for DOS
30  - run_dos(): Run DOS post-processing via 'dos.x'
31  - run_pdos(): Run PDOS post-processing via 'projwfc.x'
32
33  Requirements:
34  -----
35  - ASE
36  - Quantum ESPRESSO (pw.x, dos.x, projwfc.x)
37  - NumPy
38  - Pseudopotentials from PSLibrary
39
40  """
41
42  # file: gaas_workflow.py
43
44  import os
45  import numpy as np
46  from ase.build import bulk
47  from ase.calculators.espresso import Espresso, EspressoProfile
48  from ase.dft.kpoints import bandpath
```

```

49 import xml.etree.ElementTree as ET
50
51 def create_structure():
52     return bulk('GaAs', crystalstructure='zincblende', a=5.65)
53
54 def run_scf(atoms, profile):
55     scf_calc = Espresso(
56         profile=profile,
57         input_data={
58             'control': {'calculation': 'scf', 'prefix': 'gaas', 'outdir':
59                 './out'},
60             'system': {
61                 'ecutwfc': 40, 'ecutrho': 320,
62                 'occupations': 'smearing', 'smearing': 'gaussian', '
63                 degauss': 0.01
64             },
65             'electrons': {'conv_thr': 1e-6}
66         },
67         pseudopotentials={
68             'Ga': 'Ga.pbe-dn-rrkjus_psl.1.0.0.UPF',
69             'As': 'As.pbe-n-rrkjus_psl.1.0.0.UPF'
70         },
71         kpts=(8, 8, 8)
72     )
73     atoms.calc = scf_calc
74     energy = atoms.get_potential_energy()
75     print(f"[SCF] Total energy: {energy:.6f} eV")
76
77 def run_band_structure(atoms, profile):
78     path = bandpath('GXWKGLUWLK', atoms.cell, npoints=100)
79     kpts_bands = np.zeros((len(path.kpts), 4))
80     kpts_bands[:, :3] = path.kpts
81
82     bands_calc = Espresso(
83         profile=profile,
84         input_data={
85             'control': {'calculation': 'bands', 'prefix': 'gaas', 'outdir':
86                 './out'},
87             'system': {
88                 'ecutwfc': 40, 'ecutrho': 320,
89                 'occupations': 'smearing', 'smearing': 'gaussian', '
90                 degauss': 0.01
91             },
92             'electrons': {'conv_thr': 1e-6}
93         },
94         pseudopotentials={
95             'Ga': 'Ga.pbe-dn-rrkjus_psl.1.0.0.UPF',
96             'As': 'As.pbe-n-rrkjus_psl.1.0.0.UPF'
97         },
98         kpts=kpts_bands,
99         koffset=False
100     )
101     atoms.calc = bands_calc
102     atoms.calc.calculate(atoms, properties=['bands'], system_changes=['

```

```

    cell'])
99     print("[Bands] Band structure calculation completed.")
100
101 def run_nscf(atoms, profile):
102     nscf_calc = Espresso(
103         profile=profile,
104         input_data={
105             'control': {'calculation': 'nscf', 'prefix': 'gaas', 'outdir':
106                         './out'},
107             'system': {
108                 'ecutwfc': 40, 'ecutrho': 320,
109                 'occupations': 'smearing', 'smearing': 'gaussian', '
110                 degauss': 0.01
111             },
112             'electrons': {'conv_thr': 1e-6}
113         },
114         pseudopotentials={
115             'Ga': 'Ga.pbe-dn-rrkjus_psl.1.0.0.UPF',
116             'As': 'As.pbe-n-rrkjus_psl.1.0.0.UPF'
117         },
118         kpts=(12, 12, 12)
119     )
120     atoms.calc = nscf_calc
121     atoms.calc.calculate(atoms, properties=['energy'], system_changes=['
122     cell'])
123     print("[NSCF] NSCF calculation completed.")
124
125 def run_dos():
126     with open('dos.in', 'w') as f:
127         f.write("""
128 Edos
129     outdir = './out',
130     prefix = 'gaas',
131     fildos = 'gaas.dos',
132     emin = -15.0,
133     emax = 15.0,
134     deltae = 0.01,
135 /
136     """)
137     os.system('dos.x < dos.in > dos.out')
138     print("[DOS] DOS calculation completed.")
139
140 def run_pdos():
141     with open('projwfc.in', 'w') as f:
142         f.write("""
143 Eprojwfc
144     outdir = './out',
145     prefix = 'gaas',
146     filpdos = 'gaas.pdos',
147     emin = -15.0,
148     emax = 15.0,
149     deltae = 0.01,
150     ngauss = 0,
151     degauss = 0.01,

```

```

149 /
150 """
151     os.system('projwfc.x < projwfc.in > projwfc.out')
152     print("[PDOS] PDOS calculation completed.")
153
154 def main():
155     pseudo_dir = '/yourpath'
156     profile = EspressoProfile(command='pw.x', pseudo_dir=pseudo_dir)
157     atoms = create_structure()
158
159     run_scf(atoms, profile)
160     run_band_structure(atoms, profile)
161     run_nscf(atoms, profile)
162     run_dos()
163     run_pdos()
164
165 if __name__ == "__main__":
166     main()

```

Listing 1: GaAs Electronic Structure Calculation Workflow using ASE + QE

```

1  """
2  =====
3  Charge Density Visualization Workflow from CUBE File
4  =====
5
6  Author   : Student Agent
7  Date      : July 20, 2025
8  Version   : 1.0
9  Language  : Python 3.x
10
11 Description:
12 -----
13 This script reads charge density data from a Gaussian CUBE file and
14 generates
15 combined 2D slice and 3D isosurface visualizations using matplotlib and
16 scikit-image.
17
18 Key Features:
19 - Robust parsing of CUBE format (including atomic metadata and volumetric
20   grid)
21 - Automatic handling of mismatched or missing voxel data
22 - Visualization includes:
23   - XY-plane density slice (log scale)
24   - 3D charge isosurfaces with atomic positions
25   - Lattice vectors
26
27 Requirements:
28 -----
29 - NumPy
30 - Matplotlib
31 - scikit-image
32
33 Usage:

```



```

31 -----
32 Make sure a valid 'charge_density.cube' file is in the working directory.
33 Run this script directly to generate 'combined_charge_density_optimized.
   svg'.
34 """
35
36 import os
37 import numpy as np
38 import matplotlib.pyplot as plt
39 from matplotlib.colors import LogNorm
40 from mpl_toolkits.axes_grid1 import make_axes_locatable
41 from skimage.measure import marching_cubes
42
43
44 def read_cube_file(filename):
45     """Read and parse charge density from a CUBE file."""
46     with open(filename, 'r') as f:
47         # Skip comment lines
48         f.readline()
49         f.readline()
50
51         # Read number of atoms and origin
52         line = f.readline().split()
53         natoms = int(line[0])
54         origin = np.array([float(x) for x in line[1:4]])
55
56         # Read voxel grid and spacing
57         nx, dx = int(f.readline().split()[0]), np.array([float(x) for x in
   f.readline().split()[1:4]])
58         ny, dy = int(f.readline().split()[0]), np.array([float(x) for x in
   f.readline().split()[1:4]])
59         nz, dz = int(f.readline().split()[0]), np.array([float(x) for x in
   f.readline().split()[1:4]])
60
61         # Read atomic information
62         atoms = []
63         for _ in range(abs(natoms)):
64             parts = f.readline().split()
65             atomic_number = int(parts[0])
66             charge = float(parts[1])
67             position = np.array([float(x) for x in parts[2:5]])
68             atoms.append((atomic_number, charge, position))
69
70         # Read all remaining density values
71         data = np.zeros((nx, ny, nz))
72         raw_values = []
73         for line in f:
74             raw_values.extend([float(val) for val in line.split()])
75
76         expected = nx * ny * nz
77         if len(raw_values) != expected:
78             print(f"[Warning] Data size mismatch: expected {expected}, got
   {len(raw_values)}")
79             raw_values = raw_values[:expected]

```

```

80     # Populate 3D array
81     idx = 0
82     for i in range(nx):
83         for j in range(ny):
84             for k in range(nz):
85                 data[i, j, k] = raw_values[idx] if idx < len(
86                     raw_values) else 0.0
87                 idx += 1
88
89     return data, atoms, origin, (dx, dy, dz)
90
91
92 def create_combined_plot(data, atoms_info, origin, grid_vectors):
93     """Create combined 2D slice and 3D isosurface visualization."""
94     print("[Info] Generating visualizations...")
95     plt.figure(figsize=(20, 8))
96
97     # Log scale normalization
98     vmin = data[data > 0].min()
99     vmax = data.max()
100    norm = LogNorm(vmin=vmin, vmax=vmax)
101
102    # ----- 2D Slice -----
103    ax1 = plt.subplot(122)
104    slice_xy = data[:, :, data.shape[2] // 2]
105    im = ax1.imshow(slice_xy.T, cmap='viridis', origin='lower', norm=norm)
106    ax1.set_title('XY Plane Charge Density Slice')
107    ax1.set_xlabel('X (Grid Points)')
108    ax1.set_ylabel('Y (Grid Points)')
109
110    divider = make_axes_locatable(ax1)
111    cax = divider.append_axes("right", size="5%", pad=0.1)
112    plt.colorbar(im, cax=cax, label='Charge Density (e/Bohr3)')
113
114    # ----- 3D Isosurfaces -----
115    ax2 = plt.subplot(121, projection='3d')
116    levels = np.logspace(np.log10(vmin), np.log10(vmax), 4)[1:3]
117    colors = plt.cm.viridis(norm(levels))
118
119    for i, level in enumerate(levels):
120        try:
121            verts, faces, _, _ = marching_cubes(data, level)
122            ax2.plot_trisurf(verts[:, 0], verts[:, 1], faces, verts[:, 2],
123                            color=colors[i], alpha=0.4 + i * 0.2, lw=0.5)
124        except Exception as e:
125            print(f"[Warning] Skipping isosurface at level {level:.2e}: {e}")
126
127    # ----- Atoms -----
128    element_colors = {31: 'blue', 33: 'red'} # Ga and As
129    element_labels = {31: 'Ga', 33: 'As'}
130    added = set()

```

```

132     for atomic_num, _, pos in atoms_info:
133         color = element_colors.get(atomic_num, 'gray')
134         label = element_labels.get(atomic_num, 'Unknown') if atomic_num
            not in added else None
135         ax2.scatter(*pos, c=color, s=150, edgecolor='k', depthshade=False,
            label=label)
136         added.add(atomic_num)
137
138     # ----- Lattice Vectors -----
139     for i, vec in enumerate(grid_vectors):
140         ax2.quiver(*origin, *(vec * data.shape[i]), color='black',
            arrow_length_ratio=0.1, lw=2)
141
142     ax2.set_xlabel('X (Bohr)', labelpad=15)
143     ax2.set_ylabel('Y (Bohr)', labelpad=15)
144     ax2.set_zlabel('Z (Bohr)', labelpad=15)
145     ax2.zaxis._axinfo['juggled'] = (1, 2, 0)
146     ax2.set_title('3D Charge Density Isosurfaces')
147     ax2.legend(title='Atoms', loc='upper right', bbox_to_anchor=(1.15,
        0.9))
148     ax2.view_init(elev=30, azim=45)
149
150     plt.savefig('combined_charge_density_optimized.svg', dpi=300,
        bbox_inches='tight')
151     plt.show()
152
153
154 if __name__ == '__main__':
155     # Load and visualize charge density from CUBE file
156     cube_file = 'charge_density.cube'
157     if not os.path.isfile(cube_file):
158         raise FileNotFoundError(f"CUBE file not found: {cube_file}")
159
160     charge_data, atoms_info, origin, grid_vectors = read_cube_file(
        cube_file)
161     print(f"[Info] Charge density shape: {charge_data.shape}")
162     create_combined_plot(charge_data, atoms_info, origin, grid_vectors)

```

Listing 2: GaAs Charge Density Visualization

Numerical Methods and Parameters

The thin film evolution on the window was solved using a finite difference method on a rectangular grid of $nx = 400$ and $ny = 200$, with spatial resolutions $dx = L/nx = 0.0025$ m and $dy = W/ny = 0.0025$ m. The time step was set to $dt = 0.0002$ s, resulting in a total of $nt = T/dt = 50000$ steps.

The Laplacian operator ∇^2 was constructed as a sparse matrix:

$$D_{2x} = \frac{1}{dx^2} \begin{pmatrix} 1 & -2 & 1 \end{pmatrix}, \quad \nabla^2 = I_y \otimes D_{2x} + D_{2y} \otimes I_x,$$

where I_x and I_y are identity matrices of appropriate sizes. The flux divergence was computed using central differences, and boundary conditions were either periodic or zero-flux, implemented

implicitly via truncation.

Flow computation was restricted to regions where $h > h_{\text{min_move}} = 1 \times 10^{-5}$ m to avoid numerical diffusion. Visualization was performed using Matplotlib animations, displaying $\sqrt{h + 5 \times 10^{-5} |\nabla h|}$ to highlight the film edges.

The physical and numerical parameters used in the simulation are summarized in Table 3.

Table 3: Physical and numerical parameters

Parameter	Value
μ (viscosity)	$1 \times 10^{-3} \text{ Pa} \cdot \text{s}$
ρ (density)	1000 kg/m^3
g (gravity)	9.81 m/s^2
σ (surface tension)	0.036 N/m
τ_{air} (air shear stress)	36600 Pa
$L \times W$ (window size)	$1.0 \times 0.5 \text{ m}$
T (simulation time)	10 s
<i>branch_prob</i> (branching probability)	0.85
<i>drop_thresh</i> (dripping threshold)	$8 \times 10^{-2} \text{ m}$
<i>evap_rate</i> (evaporation rate)	$1 \times 10^{-4} \text{ m/s}$
nx, ny (grid points)	$400, 200$
dt (time step)	0.0002 s